GSE Management Unit (GSEMU)

Hardware and Software Reference

Ken Overton

KJO Liquid Rocket Motor Control System

Version: 0.4
Date: March 2026

This document describes the hardware, software architecture, Fill valve control, and GPIO handshake protocol of the GSE Management Unit. The GSEMU is mounted on the launch pad and controls the propellant fill valve and the umbilical quick-release servo.

© 2025–2026 Ken Overton. All rights reserved.

Contents

1 Hardware Overview **2**
1.1 Component Summary . . . 2
1.2 MCP23X17 GPIO Expander Pin Map . . . 2
1.3 Hardware Block Diagram . . . 3

2 Software Architecture **3**
2.1 Module Summary . . . 3
2.2 Class Hierarchy . . . 3
2.3 Main Loop Control Flow . . . 4

3 Fill Valve Control (`Slave_Valve`) **4**
3.1 Slave_Valve Operation . . . 4
3.2 Local Button Override . . . 4
3.3 Handshake from GSEMU Perspective . . . 5

4 Umbilical Quick-Release (`QR_Slave`) **5**
4.1 Overview . . . 5
4.2 Hardware . . . 5
4.3 Signal Protocol . . . 5
4.4 Level-Follower Operation . . . 6
4.5 Local Button A Override . . . 6
4.6 QR_Slave API . . . 6

5 OLED Display **6**

6 SD Logging **7**

7 Version History **7**

1 Hardware Overview

1.1 Component Summary

Component	Part	Interface	Notes
Microcontroller	Adafruit Feather M0 (ATSAMD21G18A, 48 MHz)	—	
OLED Display	SH1107, 128×64, 1.12"	I^2C	
GPIO Expander	MCP23X17 (16 I/O)	I^2C	Buttons, AUX_IO, buzz
PWM Servo Wing	Adafruit 8-channel wing (PCA9685)	I^2C	Fill valve servo
Real-Time Clock	PCF8523	I^2C	
SD Card	Standard SPI SD socket	SPI	
Buzzer	Passive piezo via MCP23X17	GPIO	
Battery	3.7 V LiPo	—	

1.2 MCP23X17 GPIO Expander Pin Map

Pin Name	Direction	Function
Button A (QR release)	IN	Hold-to-release QR latch (servo open while held, hold when rel
Button B (fill close)	IN	Locally command Fill valve closed
Button C (fill open)	IN	Locally command Fill valve open
`AUX_IO_1` (CMD)	IN	Fill valve CMD from EMU
`AUX_IO_2` (STATUS)	OUT	Fill valve STATUS to EMU
`AUX_IO_3` (`QR_CMD_EIO`)	IN	Umbilical QR release CMD from EMU (INPUT_PULLUP; LO
`AUX_IO_4` (`RELEASE_STATE_EIO`)	OUT	Umbilical separation state; wired to GND (LOW = connected,
`AUX_IO_5` (`LAUNCH_ENABLE_EIO`)	IN	Remote start enable from EMU (INPUT_PULLUP; LOW = e
`AUX_IO_6` (`REMOTE_START_EIO`)	OUT	Remote start command to EMU (OUTPUT; LOW = start con
Buzzer output	OUT	Audible alert

1.3 Hardware Block Diagram

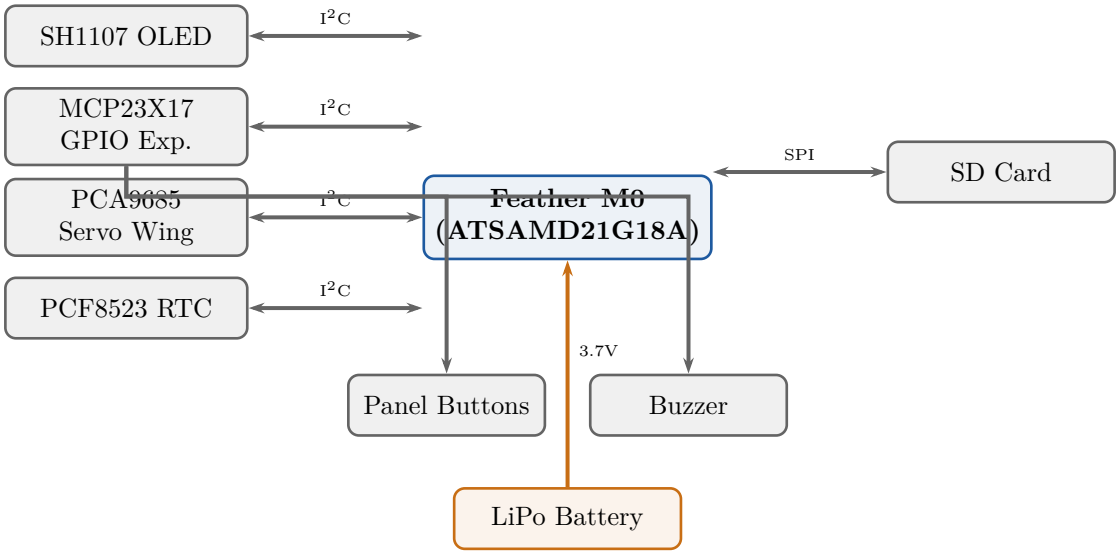

Figure 1: GSEMU hardware block diagram.

2 Software Architecture

2.1 Module Summary

Module	Location	Responsibility
`main.cpp`	local	Setup, loop, button handling
`KJO_Valve`	local	Base valve: servo drive, pot feedback
`KJO_Slave_Valve`	local	CMD/STATUS GPIO handshake, local override
`KJO_QR_Servo`	local	Servo-only actuator base class (time-based position)
`KJO_QR_Slave`	local	QR release level-follower; CMD (`AUX_IO_3`), state (`AUX_IO_4`), local overrid
`KJO_GPIO`	local	MCP23X17 pin assignments, QR servo constants
`KJO_GSE_Display`	local	SH1107 OLED display, SD log helpers
`KJO_Logging`	local	SD log file constants
`KJO_System_Config`	shared	Version, platform constants

2.2 Class Hierarchy

`Valve` (base) ⟵ `Slave_Valve` (derived).

The `Valve` base class provides servo drive and potentiometer feedback. `Slave_Valve` adds edge-detection polling of `AUX_IO_1` (CMD) and output driving of `AUX_IO_2` (STATUS).

`QR_Servo` (base) ⟵ `QR_Slave` (derived).

`QR_Servo` provides servo drive via the PCA9685 with time-based motion completion detection (no potentiometer). `QR_Slave` adds level-based CMD following (`AUX_IO_3`: LOW = open latch, HIGH = hold latch), physical separation detection via `AUX_IO_4` (`RELEASE_STATE_EIO`), and a local Button A override (`localRelease()` / `localHold()`).

2.3 Main Loop Control Flow

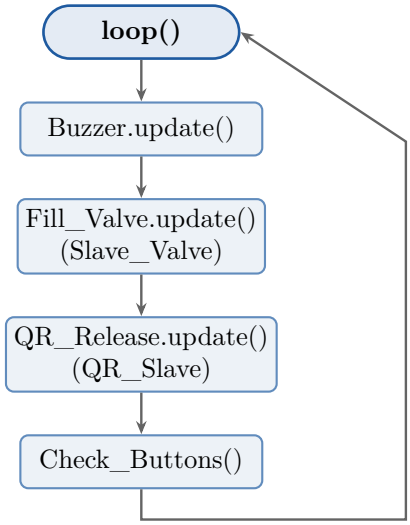

Figure 2: GSEMU main loop control flow.

3 Fill Valve Control (`Slave_Valve`)

3.1 Slave_Valve Operation

`Slave_Valve.update()` is called every loop iteration. It performs edge detection on the CMD line (`AUX_IO_1`) and responds as follows:

CMD transition	Slave_Valve action	STATUS (`AUX_IO_2`)
LOW → HIGH (rising edge)	Drive servo to fully open	Asserted HIGH when open
HIGH → LOW (falling edge)	Drive servo to fully closed	Asserted HIGH when closed
No change	Poll servo position	Unchanged

STATUS is asserted as soon as the potentiometer confirms the valve has reached the commanded position (within dead-band).

3.2 Local Button Override

Three panel buttons on the MCP23X17 provide local control during ground operations:

Button	Action
A (QR Release)	**Hold-to-release**: calls `localRelease()` on press; servo opens while button held. Calls `lo`
B (Fill Close)	Drive Fill valve servo to fully closed position (falling-edge triggered).
C (Fill Open)	Drive Fill valve servo to fully open position (falling-edge triggered).

Warning

Button A activates the umbilical quick-release servo. The latch opens for as long as the button is held, and releases pressurised N_2O when the umbilical separates. Ensure all personnel are clear and the pad is safe before pressing Button A. Buttons B and C bypass EMU command authority; use for ground testing only.

3.3 Handshake from GSEMU Perspective

The sequence from the GSEMU point of view:

1. Poll `AUX_IO_1`: if rising edge detected, enter OPENING state.
2. Drive servo toward fully open; `Valve::update()` monitors pot.
3. When `isOpen()` returns true, assert `AUX_IO_2` = HIGH.
4. Poll `AUX_IO_1`: if falling edge detected, enter CLOSING state.
5. Drive servo to closed; assert `AUX_IO_2` = HIGH when complete.
6. De-assert `AUX_IO_2` after a brief confirmation hold.

4 Umbilical Quick-Release (`QR_Slave`)

4.1 Overview

The umbilical quick-release servo physically separates the umbilical connector from the rocket airframe at launch. The GSEMU drives the servo; the EMU commands the release via a level-based GPIO protocol on two AUX connector lines. The `QR_Slave` class manages the servo, the CMD-level follower, and physical separation detection.

4.2 Hardware

The QR servo is driven by a channel on the PCA9685 Servo Wing. Unlike the Fill valve servo, there is no potentiometer feedback; position is determined entirely by elapsed time (`QR_SERVO_MOVE_MS`).

Two MCP23X17 expansion I/O lines are used:

- `AUX_IO_3` (`QR_CMD_EIO`) — INPUT_PULLUP. Reads the CMD level from the EMU.
- `AUX_IO_4` (`RELEASE_STATE_EIO`) — wired to GSEMU chassis GND. When read by the EMU as INPUT_PULLUP: LOW = umbilical connected, HIGH = connector removed (separated).

Warning

`QR_SERVO_PWM_HOLD` and `QR_SERVO_PWM_OPEN` are placeholder values in the current firmware. They **must** be calibrated on the bench before any live operation. Verify servo travel direction and endpoint positions before connecting the umbilical.

4.3 Signal Protocol

`AUX_IO_3` (`QR_CMD_EIO`) is driven by the EMU (`QR_Master`). The GSEMU reads it with INPUT_PULLUP on every `update()` call:

QR_CMD_EIO Level	Meaning	Servo action
HIGH (default)	Latch hold / armed	Servo commanded to HOLD position
LOW	Release commanded	Servo commanded to OPEN position

When the AUX connector is disconnected, INPUT_PULLUP floats HIGH — the servo defaults to HOLD (safe, latch engaged).

4.4 Level-Follower Operation

`QR_Slave::update()` is called every `loop()` iteration. It reads the current level of `QR_CMD_EIO` (or uses the local override flag if Button A is held) and drives the servo accordingly:

- CMD LOW and servo not already at or moving to OPEN ⇒ call `openServo()`.
- CMD HIGH and servo not already at or moving to HOLD ⇒ call `holdServo()`.

The servo holds its last commanded position while the CMD level is unchanged. There is no terminal/latch state: the servo will return to HOLD any time the CMD line goes HIGH (e.g., EMU calls `hold()`, power cycle, or AUX cable disconnection).

4.5 Local Button A Override

Button A on the GSEMU front panel engages a local override that bypasses the CMD line. `Check_Buttons()` calls:

- `QR_Release.localRelease()` on button press (falling edge).
- `QR_Release.localHold()` on button release (rising edge).

While the override is active (`_local_override = true`), `update()` substitutes LOW for the CMD pin read — the servo stays open regardless of what the EMU is driving. The override is cleared on button release; the servo returns to CMD-line following on the next `loop()` iteration.

4.6 QR_Slave API

Method	Action
`begin()`	Configure CMD and state pins; command servo to HOLD
`update()`	Poll CMD (or override), drive servo; detect separation
`isSeparated()`	Returns true (latched) once state line has gone HIGH
`isConnected()`	Returns true while state line reads LOW
`localRelease()`	Set local override; servo opens on next `update()`
`localHold()`	Clear local override; servo follows CMD on next `update()`
`isLocalOverride()`	Returns true while Button A override is active

5 OLED Display

The SH1107 OLED provides local status when no serial console is available. `KJO_GSE_Display` provides:

- `scrollMessage(msg)`: scrolls a text message across the display.
- `displayValveStatus(pct)`: shows the current valve open percentage.
- `displayLogEntry(msg)`: echoes SD log entries to the display.

6 SD Logging

Log files are stored in the root of the SD card as `Log_N.txt` (N is auto-incremented to avoid overwriting existing files). Each record is time-stamped (PCF8523) and logs CMD/STATUS transitions, servo position changes, button presses, and system events.

7 Version History

Version	Date	Notes
0.1	March 2026	Initial release.
0.2	March 2026	Add umbilical QR release subsystem (QR_Servo, QR_Slave, AUX_IO_3 protocol).
0.3	March 2026	Level-based QR protocol (CMD LOW=release, HIGH=hold); add RELEASE_STAT
0.4	March 2026	SD log files moved to SD card root (`Log_N.txt`); subdirectory path constant (`LOG_FI`